Distributed Systems Design Document

C22789271 John Maguire

19th November 2025

"I declare that this work, which is submitted as part of

my coursework, is entirely my own, except where clearly and explicitly stated."

Application Architecture

The architecture I chose is a TCP socket approach in Python. The marketplace consists of one Seller and one or more Buyers. The seller functions as the market controller which is responsible for keeping track of the trading stock, notifying buyers of stock and timing, and managing the sales. Buyers are independent and each run in a sperate terminal. They communicate with the seller using a client-server pattern with the Buyer sending text commands over persistent socket connections.

The Seller acts as the server and its architecture is composed of 3 components: the Seller Node, Sales Timer and the Seller server. The Seller Node handles item selection and stock tracking. It also decides when to switch out items and when to close the Marketplace. The Sales Timer has a dedicated thread that triggers the expiration of items after 60 seconds of being on the market. The Seller Server manages socket connections; handles buy requests and broadcasts updates to all the buyers.

The Seller's networking layer opens a TCP server socket on a specified IP and port and uses socket.accept() to register new Buyer connections. For each connected Buyer, a dedicated thread is spawned to process their incoming messages. This ensures concurrency: multiple Buyers can interact with the Seller simultaneously without blocking behaviour.

Buyers contain two main components: The BuyerNode and the BuyerClient interface. The BuyerNode constructs buy messages and maintains the buyers ID. The BuyerClient handles network communication and user activity. This features a listener thread that allows the buyers to receive messages at any time of new items, updates on stock and when the market is closed.

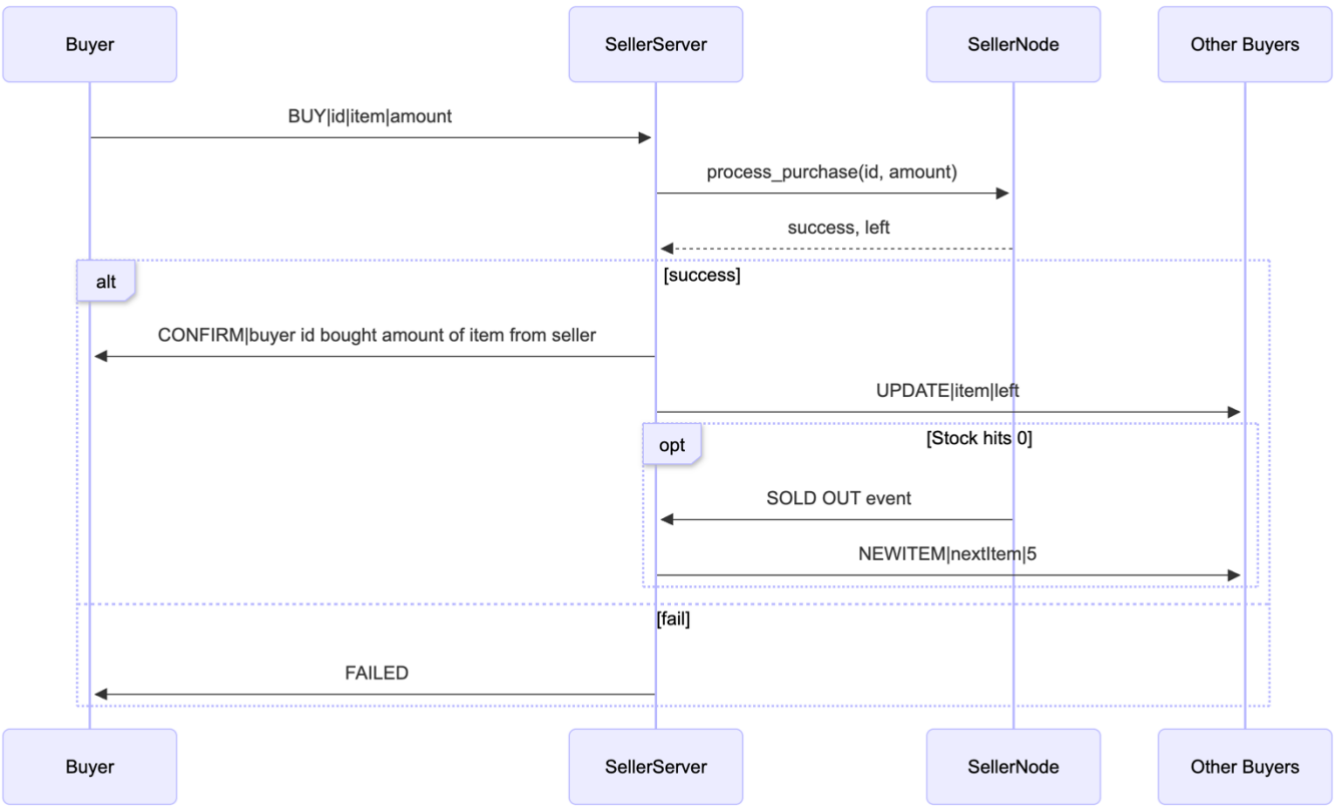
Buyers act as TCP clients that initiate a connection to the Seller. When a Buyer connects, the Seller maintains the TCP connection for the entire session and stores the associated socket in an internal connection list. While they are connected each Buyer runs both input and output. The main Buyer thread handles input and sends List/Buy commands to the Seller while the background listener thread receives the server messages continuously. This ensures that Sellers can push real-time events, like

updates on stock, to the Buyers without interrupting the Buyers' main interactive prompt.

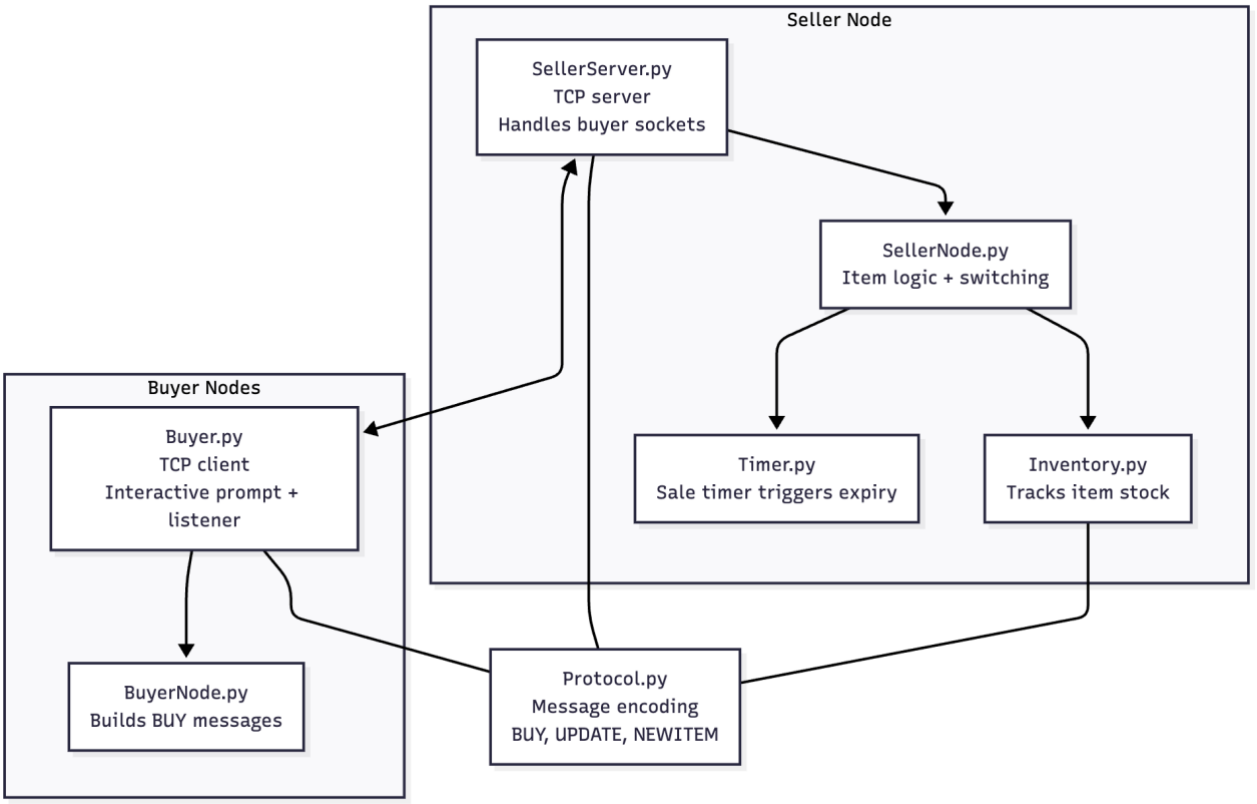
Communication is a simple text-based protocol. The messages (LIST, BUY, CONFIRM, UPDATE, NEWITEM, TIMELEFT, WRONGITEM, ERROR, and MARKETCLOSED) are encoded as separated strings e.g. BUY|2|sugar|. This protocol ensures consistency in formatting and encoding across all parts of the system as each node uses UTF-8 encoding.

The system's is organised so that each major component is isolated into its own Python file: seller logic, protocol definitions, buyer logic, inventory handling, and timers. This separation provides clarity as each file handles one part of the system not only making development easier but also making it simpler to explain, test, and demonstrate in a lab setting. Because everything is built entirely with Python's built-in socket and threading libraries, it runs without modification on Windows, Linux, or macOS and does not rely on containers or additional installation steps.

Sequence Diagram



Architecture Diagram



Directory Structure

Assignment/

├── buyer.py	Main Buyer client, reads user commands, receives messages
├── buyer_node.py	Creates encoded BUY messages for the seller
├── seller.py	Starts TCP server, accepts buyers, processes LIST/BUY and updates.
├── seller_node.py	Manages item lifecycle, sold-out tracking, switching items.
├── inventory.py	Tracks stock for each item.
├── timer.py	Runs a countdown; calls SellerNode when time expires.
├── protocol.py	Defines message format, commands, item list, timer duration
├── config.cfg	config values for ports/timers
└── README.md	Documentation & run instructions

How To Run

1. Check Python version 3.8+ is installed using **python --version**
2. Navigate to the project and open a command prompt
3. Start the seller with **python seller.py.** You should see output similar to
   [SellerNode] Now selling: oil (5 units)
   [Seller] Running on 127.0.0.1:6010
4. Now open a new command prompt. Each new terminal is another buyer.
5. Start a buyer with **python buyer.py** and you will be assigned a buyer ID. You should now see:
   [INFO] Assigned Buyer ID = 1

   [SIGNAL] NEWITEM|potato|5

   [SIGNAL] TIMELEFT|potato|60
   Enter command (list/buy/exit):

   Run multiple buyers by opening multiple terminals using different IDs for each.
6. As a buyer you can LIST the item for sale e.g.
   Enter command (list/buy/exit): list
   Enter command (list/buy/exit):
   [SIGNAL] ITEM|potato|5
7. You can enter the buy option e.g.
   Enter command (list/buy/exit): buy
   Item: sugar
   Amount: 1
   Enter command (list/buy/exit):
   [SIGNAL] UPDATE|sugar|4
   [SIGNAL] CONFIRM|buyer 1 bought 1 of sugar from seller

   When a purchase is made all active buyers will be sent an update of the new stock. The buyer that bought the item will receive a confirmation of the purchase.
8. If the stock is not sold out by the 60 second timer a new item will be put on the market and all buyers will be notified e.g.
   [SIGNAL] NEWITEM|oil|5
   [SIGNAL] TIMELEFT|oil|60
9. The buyer can also exit which disconnects the buyer e.g.
   Enter command (list/buy/exit): exit
   (base) johnmaguire@MacBookPro/Assignment %
10. To stop the seller press **CTRL + C** in the seller window

Flow example of one buyer 3 sellers

```
(base) johnmaguire@MacBookPro Assignment %
python seller.py
[SellerNode] Now selling: flower (5 units)
[Seller] Running on 127.0.0.1:6010
[Seller] Buyer connected: ('127.0.0.1', 491
81), assigned ID = 1
[Seller] Buyer connected: ('127.0.0.1', 491
82), assigned ID = 2
[Seller] Buyer connected: ('127.0.0.1', 491
83), assigned ID = 3
buyer 1 bought 2 of flower from SELLER
buyer 2 bought 3 of flower from SELLER
[SellerNode] SOLD OUT: flower cannot be sol
d again.
[SellerNode] Now selling: oil (5 units)
buyer 3 bought 2 of oil from SELLER
[SellerNode] Time expired -> Switching item
[SellerNode] Now selling: sugar (5 units)
buyer 1 bought 5 of sugar from SELLER
[SellerNode] SOLD OUT: sugar cannot be sold
 again.
[SellerNode] Now selling: potato (5 units)
buyer 2 bought 5 of potato from SELLER
[SellerNode] SOLD OUT: potato cannot be sol
d again.
[SellerNode] Now selling: oil (5 units)
buyer 3 bought 3 of oil from SELLER
[SellerNode] SOLD OUT: oil cannot be sold a
gain.
[SellerNode] ALL ITEMS SOLD OUT — Marketpla
ce closing.
[Seller] Broadcasted MARKET CLOSED to all b
uyers.
```

```
[SIGNAL] TIMELEFT|flower|60
buy
Item: flower
Amount: 2
Enter command (list/buy/exit):
[SIGNAL] UPDATE|flower|3

[SIGNAL] CONFIRM|buyer 1 bought 2 of flower
 from seller

[SIGNAL] NEWITEM|oil|5

[SIGNAL] TIMELEFT|oil|60

[SIGNAL] UPDATE|flower|0

Enter command (list/buy/exit):
[SIGNAL] UPDATE|oil|3

Enter command (list/buy/exit):
[SIGNAL] TIMEEXPIRED|oil

[SIGNAL] NEWITEM|sugar|5

[SIGNAL] TIMELEFT|sugar|60

Enter command (list/buy/exit): buy
Item: sugar
Amount: 5
Enter command (list/buy/exit):
[SIGNAL] NEWITEM|potato|5

[SIGNAL] TIMELEFT|potato|60

[SIGNAL] UPDATE|sugar|0

[SIGNAL] CONFIRM|buyer 1 bought 5 of sugar
from seller

Enter command (list/buy/exit):
[SIGNAL] NEWITEM|oil|3

[SIGNAL] TIMELEFT|oil|60

[SIGNAL] UPDATE|potato|0

Enter command (list/buy/exit):
[SIGNAL] MARKETCLOSED

[SIGNAL] UPDATE|oil|0

Enter command (list/buy/exit):
```

```
[SIGNAL] UPDATE|flower|3

Enter command (list/buy/exit): buy
Item: flower
Amount: 3
Enter command (list/buy/exit):
[SIGNAL] NEWITEM|oil|5

[SIGNAL] TIMELEFT|oil|60

[SIGNAL] UPDATE|flower|0

[SIGNAL] CONFIRM|buyer 2 bought 3 of flower
 from seller

Enter command (list/buy/exit):
[SIGNAL] UPDATE|oil|3

Enter command (list/buy/exit):
[SIGNAL] TIMEEXPIRED|oil

[SIGNAL] NEWITEM|sugar|5

[SIGNAL] TIMELEFT|sugar|60

Enter command (list/buy/exit):
[SIGNAL] NEWITEM|potato|5

[SIGNAL] TIMELEFT|potato|60

[SIGNAL] UPDATE|sugar|0

Enter command (list/buy/exit): buy
Item: potato
Amount: 5
Enter command (list/buy/exit):
[SIGNAL] NEWITEM|oil|3

[SIGNAL] TIMELEFT|oil|60

[SIGNAL] UPDATE|potato|0

[SIGNAL] CONFIRM|buyer 2 bought 5 of potato
 from seller

Enter command (list/buy/exit):
[SIGNAL] MARKETCLOSED

[SIGNAL] UPDATE|oil|0

Enter command (list/buy/exit):
```

```
[SIGNAL] NEWITEM|oil|5

[SIGNAL] TIMELEFT|oil|60

[SIGNAL] UPDATE|flower|0

Enter command (list/buy/exit): buy
Item: oil
Amount: 2
Enter command (list/buy/exit):
[SIGNAL] UPDATE|oil|3

[SIGNAL] CONFIRM|buyer 3 bought 2 of oil
from seller

Enter command (list/buy/exit):
[SIGNAL] TIMEEXPIRED|oil

[SIGNAL] NEWITEM|sugar|5

[SIGNAL] TIMELEFT|sugar|60

Enter command (list/buy/exit):
[SIGNAL] NEWITEM|potato|5

[SIGNAL] TIMELEFT|potato|60

[SIGNAL] UPDATE|sugar|0

Enter command (list/buy/exit):
[SIGNAL] NEWITEM|oil|3

[SIGNAL] TIMELEFT|oil|60

[SIGNAL] UPDATE|potato|0

Enter command (list/buy/exit): nuy
Enter command (list/buy/exit): oil
Enter command (list/buy/exit): 5
Enter command (list/buy/exit): buy
Item: oil
Amount: 3
Enter command (list/buy/exit):
[SIGNAL] MARKETCLOSED

[SIGNAL] UPDATE|oil|0

[SIGNAL] CONFIRM|buyer 3 bought 3 of oil
from seller

Enter command (list/buy/exit):
```

Alternative Design Solution

My alternative design would be instead of relying on raw TCP sockets and manual message parsing, to use a REST microservice architecture built on HTTP. The marketplace runs on a small HTTP server framework such FastAPI, and each buyer becomes a lightweight HTTP client. The Seller is implemented as a central service that exposes REST endpoints for listing items, buying items, and receiving stock updates.

The major improvement in this design is that all networking, threading, and message parsing are handled by the web framework, significantly reducing complexity. Buyers do not maintain long-running TCP connections each request is stateless, and updates can be delivered through polling or WebSocket notifications. This design is easier to scale, debug, and deploy, but was not implemented because the assignment required raw sockets only.

Description of components

Seller Service (REST API)

- Built with FastAPI.
- Maintains item inventory, timers, and sale logic.
- Provides HTTP endpoints:
    - GET /item → returns current item and stock.
    - POST /buy → attempts purchase and returns confirmation or failure.
    - GET /time → returns time remaining.
- Optionally includes a WebSocket endpoint for real-time updates.

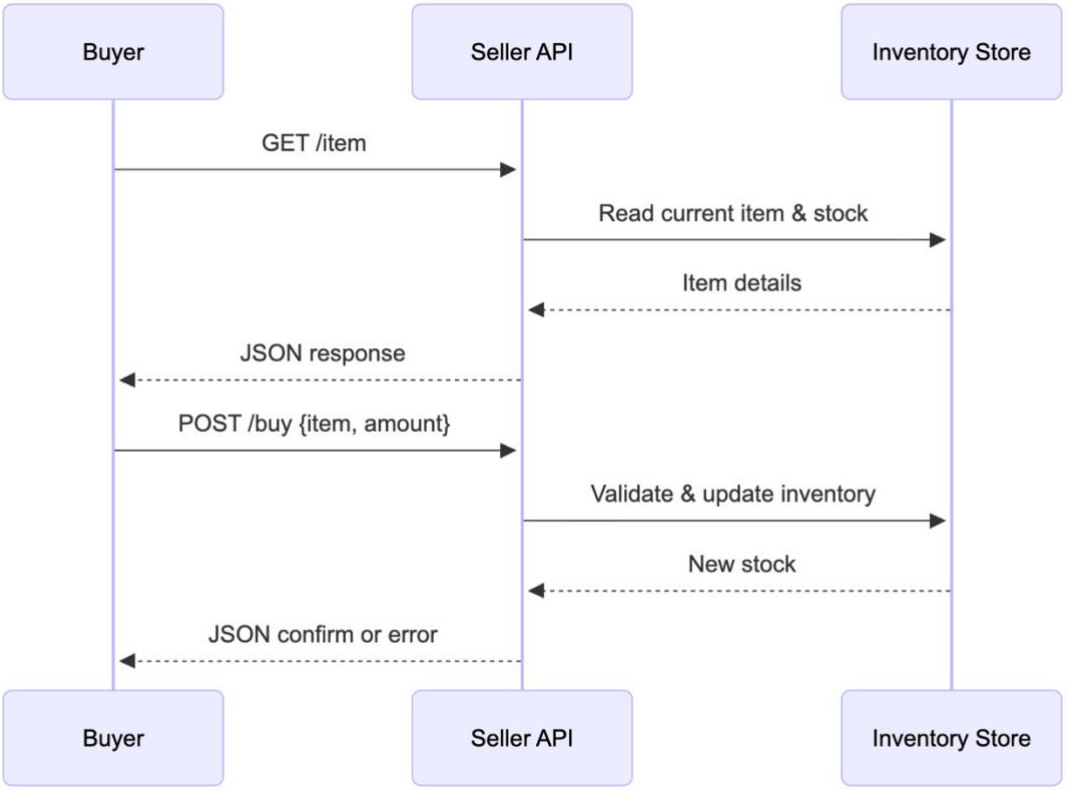
Buyer Client

- A simple Python script using requests (for HTTP) or web sockets.
- Periodically polls:
    - /item for updates
    - /time for remaining sale time
- Sends purchases via POST /buy.

Timer & Scheduler

- Implemented using apscheduler or FastAPI background tasks.

- Automatically rotates items after timeout or when sold out.

This architecture removes the need for manual socket handling and thread management. The buyer would communicate with the seller using HTTP/HTTPS. The communication is simpler, safer, and easier to test compared to raw TCP. This method is also more scalable and reduces code-complexity. The use of web frameworks and JSON would allow the browser to handle parsing automatically instead of my approach which is manual. However one disadvantage would be that it would not support real time updates and push messages. Overall it trades natural broadcast messages for a simple modern design that is scalable and easier to deploy.

Sequence Diagram